

FUND-TRANSFER ENDPOINT IMPLEMENTATION

Endpoint

POST /transfer/internal.php

Transfers funds internally between two bank accounts within the system.

Request

```
session_start();  
header("Content-Type: application/json");
```

Body Parameters (JSON)

Field	Type	Is required?	Description
recipient_account	string	yes	Recipient's account holder ID
account_holder_id	string	yes	Sender's account holder ID
amount	float	yes	Amount to transfer

Backend Logic Flow

1. It validates for required fields and amount > 0.
2. It checks for self-transfer, sender/recipient account existence, and sufficient balance.
3. It uses a transaction and performs the deduction/addition.
4. It logs the transaction.
5. It commits changes and returns a response.

Response

Success

```
echo json_encode([  
    "success" => true,  
    "message" => "Transfer completed successfully.",  
    "transaction_id" => $transactionId,  
    "amount_transferred" => number_format($amount, 2),  
    "remaining_balance" => number_format($updatedBalance, 2),  
    "recipient_account" => $recipientAccount  
]);
```

Error Responses

Invalid Input of Account or Amount (400)

```
if (empty($recipientAccount) || $amount <= 0) {  
    http_response_code(400);  
    echo json_encode(["success" => false, "error" => "Invalid recipient account or amount."]);  
    exit();  
}
```

Invalid Amount Format (400)

```
if (!is_numeric($_POST['amount']) || $amount != $_POST['amount']) {  
    http_response_code(400);  
    echo json_encode(["success" => false, "error" => "Invalid amount format."]);  
    exit();  
}
```

Sender Not Found (400)

```
if (!$senderAccount) {  
    $pdo->rollBack();  
    http_response_code(400);  
    echo json_encode(["success" => false, "error" => "Sender account not found."]);  
    exit();  
}
```

Insufficient Balance (400)

```
if ($senderAccount['balance'] < $amount) {  
    $pdo->rollBack();  
    http_response_code(400);  
    echo json_encode(["success" => false, "error" => "Insufficient balance. Available: " . number_format($senderAccount['balance'], 2)]);  
    exit();  
}
```

Recipient Not Found (400)

```
if (!$recipientAccountData) {  
    $pdo->rollBack();  
    http_response_code(400);  
    echo json_encode(["success" => false, "error" => "Recipient account not found."]);  
    exit();  
}
```

Self-Transfer Attempt (400)

```
if ($senderAccount['account_number'] == $recipientAccount) {  
    $pdo->rollBack();  
    http_response_code(400);  
    echo json_encode(["success" => false, "error" => "Cannot transfer to your own account."]);  
    exit();  
}
```

Failed to Deduct Amount (500)

```
if (!$deductResult) {  
    $pdo->rollBack();  
    http_response_code(500);  
    echo json_encode(["success" => false, "error" => "Failed to deduct amount from sender account."]);  
    exit();  
}
```

Failed to Add Amount (500)

```
if (!$addResult) {  
    $pdo->rollBack();  
    http_response_code(500);  
    echo json_encode(["success" => false, "error" => "Failed to add amount to recipient account."]);  
    exit();  
}
```

Server/Database Error (500)

```
} catch (PDOException $e) {  
    $pdo->rollBack();  
    error_log("Database error in internal transfer: " . $e->getMessage());  
    http_response_code(500);  
    echo json_encode(["success" => false, "error" => "Database error occurred. Please try again."]);  
} catch (Exception $e) {  
    $pdo->rollBack();  
    error_log("General error in internal transfer: " . $e->getMessage());  
    http_response_code(500);  
    echo json_encode(["success" => false, "error" => "An error occurred. Please try again."]);  
}
```